

Scalability & Future Plan

Date	2 July 2026
Team ID	
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	1 Mark

Scalability & Future Plan:

This document outlines how the current project solution can be scaled to handle larger user bases, increased data loads, or extended features in the future. It also captures the team's roadmap for enhancing and evolving the project beyond its current state.

Current System Limitations:

S.No	Limitation	Impact	Priority to Address (High / Medium / Low)
1	SQLite is a file-based database with no built-in network access or horizontal replication support.	Limits the application to a single-server deployment. Cannot distribute read/write load across multiple database nodes for geographically distributed users.	High
2	The XGBoost model is trained on a static historical dataset with no automated retraining pipeline.	As climate patterns shift over time, the model's prediction accuracy may degrade without periodic retraining on fresh meteorological data from recent flood events.	Medium
3	No real-time weather API integration; users must manually enter all 10 meteorological parameters.	Slows down the prediction workflow and introduces potential human data-entry errors. Not suitable for automated monitoring or bulk regional scanning.	High

Scalability Plan:

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Supports up to 50 concurrent users on a single Gunicorn worker thread with SQLite WAL mode. Verified via Locust: 73 req/s, 0% errors.	Migrate to PostgreSQL with connection pooling (PgBouncer). Deploy behind Nginx reverse proxy with multiple Gunicorn workers to support 500+ concurrent users across distributed regions.
2	Data Storage	Single SQLite database file (~2MB) storing users, weather_data, and	Transition to a managed cloud database (PostgreSQL on AWS RDS or Google Cloud SQL) with automated daily backups, point-in-time recovery, and read replicas for high availability.

		predictions tables locally on the server filesystem.	
3	Performance	Average ML inference latency of 78ms. XGBoost model loaded into RAM at startup (483KB). StandardScaler preprocessing adds negligible overhead.	Implement Redis caching for repeated identical predictions. Add Celery task queue for batch regional scanning. Pre-compute risk scores during off-peak hours for high-frequency monitoring zones.
4	Security	Bcrypt password hashing, Flask session-based RBAC with 4 user roles. HTTPS enforced on Render deployment.	Add JWT token-based authentication for API access. Implement rate limiting (Flask-Limiter) to prevent brute-force attacks. Enable two-factor authentication (2FA) for Admin and Meteorologist roles.

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 1	Integrate a live weather API (OpenWeatherMap / IMD) to auto-populate the 10 meteorological parameters based on GPS location or selected district.	Q3 2026 (Aug - Sep)	Eliminates manual data entry, enabling one-click predictions. Reduces human error and enables automated district-level scanning.
Phase 2	Build an automated model retraining pipeline using Apache Airflow. Schedule monthly retraining on updated IMD flood datasets with accuracy drift monitoring.	Q4 2026 (Oct - Dec)	Maintains prediction accuracy as climate patterns evolve. Detects model degradation early via accuracy threshold alerts.
Phase 3	Add push notification alerts (SMS via Twilio + Email via SendGrid) for high-risk flood predictions. Notify subscribed users and local authorities automatically.	Q1 2027 (Jan - Mar)	Transforms from a passive query tool to an active early warning system. Enables real-time emergency alerts without users needing to visit the web interface.
Phase 4	Deploy a mobile-responsive Progressive Web App (PWA) with offline caching. Add satellite imagery overlays using Leaflet.js for visual flood zone mapping.	Q2 2027 (Apr - Jun)	Extends reach to field workers in low-connectivity rural areas. Interactive maps improve spatial understanding of flood-prone regions.